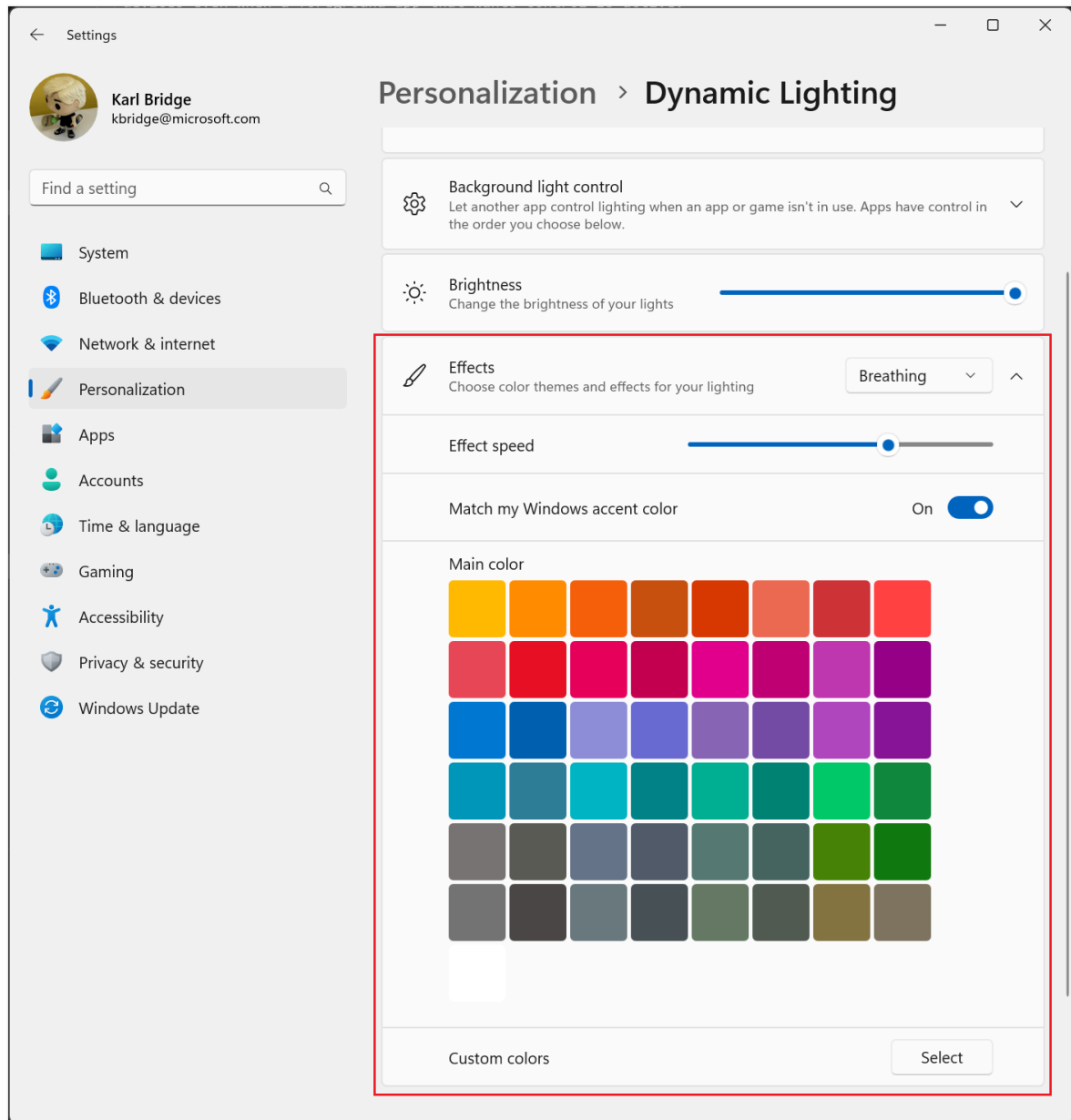


port it's connected on. If you unplug and then plug the LampArray into a different (USB) port, it will appear as a different device.

d. The *Brightness* slider lets users set the LED brightness on their devices.

e. The *Effects* dropdown lets users select colors and effects for their devices.



ⓘ Note

When a device is not selected for *Background light control*, it operates in "Autonomous mode".

This is defined in the HID spec as a mode where the hardware falls back to default behavior as defined by its firmware. For example, a device might have a preprogrammed visual effect that is the default when the OS is not actively controlling the device or if the user has opted out of OS involvement for the device.

The device must respond to the HID command to return from Autonomous mode to ensure smooth interaction with user expectations.

Packaging and app identity

Ambient background applications must declare the "com.microsoft.windows.lighting" AppExtension in the app manifest (for more detail on how to do this, see [Create and host an app extension](#)). This requirement is enforced by the AmbientLightingServer, which only accepts connections from an AmbientLightingClient in a process with package identity (packaged app) supporting the extension. This requirement is necessary to enable the user to define policy for installed apps and to then correlate that policy at runtime.

App identity is required for ambient applications such that user preferences can be determined at runtime. Once an application is installed, if it is using the ambient APIs, the system needs to correlate the running instance of an app with the user's preferences. Furthermore, making your app available to the user in the settings requires a post installation artifact that indicates to the system that your app is a legitimate user of the ambient lighting APIs.

This identity requirement is achieved via [MSIX packaging](#) (a Microsoft deployment and packaging technology formerly known as APPX).

If you are already using MSIX packaging for packaging and installation, there are no further requirements.

If you have an unpackaged app, there are additional steps required to obtain the application identity. You can either migrate your installation to full MSIX, or you can use the simplified [Sparse Packaging and External Location](#) feature of MSIX. Sparse Packaging with External Location was designed to let existing app installations gain the benefit of app identity without requiring a full conversion of the setup/install to MSIX. It is a new step in your setup/installation that uses tools to create an MSIX package to represent the app you are installing.

You must define an [AppXManifest.xml packaging manifest](#) that describes your installation. The MSIX package is created with the MakeAppXPackage tool. At install time, you install the MSIX package using a Package Manager API that specifies the location of your installed executable. For convenience, the [add-appxpackage PowerShell script](#) wraps this API behavior as well. Alternatively, [AddPackageByUriAsync](#) can be used at install time to install the MSIX package.

For unpackaged app installations, there is also a [side-by-side application manifest](#) requirement for your executable.

See the [Deployment overview](#) for a more in-depth explanation of packaged and unpackaged apps.

Examples

[LampArray sample](#)

Demonstrates how to control RGB lighting of peripheral devices using the [Windows.Devices.Lights](#) and [Windows.Devices.Lights.Effects](#) APIs.

[AutoRGB Sample](#)

Demonstrates how to extract a single, representative color from a desktop screen and use it to illuminate LED lamps on a connected RGB device.

See also

- [Windows.Devices.Lights](#)
- [Windows.Devices.Lights.Effects](#)
- [Lighting and Illumination \(www.usb.org\)](#) [↗](#)

Feedback

Was this page helpful?

[👍 Yes](#)

[👎 No](#)

[Provide product feedback](#) [↗](#) | [Get help at Microsoft Q&A](#)

Point of Service

Article • 05/06/2023

This section contains topics on how to use [Windows.Devices.PointOfService](#) to interact with peripherals found in retail and hospitality environments from Universal Windows Platform (UWP) applications.

Topic	Description
Basics	This article will help you get started with the point of service Windows Runtime APIs.
Hardware support	List of known devices that are compatible with Windows.Devices.PointOfService Windows Runtime APIs.
Barcode Scanner	Index of articles on barcode scanner.
Magnetic stripe reader	Index of articles on magnetic stripe reader.
POSPrinter	Index of articles on POSPrinter.

Point of Service basics

Article • 05/06/2023

This section contains topics that are common across all Point of Service device categories.

Topic	Description
Capability declaration	Learn how to add the pointOfService capability to your application manifest. This capability is required for use of Windows.Devices.PointOfService namespace.
Enumerating devices	Define a device selector for querying devices available to the system and use this selector to enumerate Point of Service devices.
PointOfService device objects	Learn how to create a PointOfService device object that will give you access to read-only properties of the peripheral and claim the peripheral for exclusive use in a Universal Windows Platform (UWP) application.
PointofService device claim and enable model	Use the Point of Service device claim and enable APIs to claim devices and enable them for I/O operations.
Sharing peripherals with others	Share network or Bluetooth connected peripherals with other computers in an environment where multiple PCs rely on shared peripherals rather than dedicated peripherals attached to each computer.
PointOfService end-to-end	This is an end to end example of how to interact with PointOfService peripherals utilizing the examples above.

See also

Topic	Description
Application distribution	Learn about the options for distributing your app to enterprise customers.
Application lifecycle	Learn about the life cycle of a UWP application and what happens when Windows launches, suspends, and resumes your app.
Application resources	Learn how to author, package, and consume your app's string, image, and file resources.
Data binding	Learn how to use data binding to display data in your app's UI.

Topic	Description
Device enumeration	Learn use advanced enumeration techniques to find your peripherals.
Version adaptive applications	Learn how to design your app so that it runs on multiple versions of Windows 10.

Sample code

- [Barcode scanner sample](#) 
- [Cash drawer sample](#) 
- [Line display sample](#) 
- [Magnetic stripe reader sample](#) 
- [POSPrinter sample](#) 

PointOfService device capability

Article • 05/06/2023

The PointOfService device capability is required to use APIs in the [Windows.Devices.PointOfService](#) namespace.

You request access to the PointOfService APIs by declaring the capability in your application package manifest. Most capabilities can be declared through the Manifest Designer in Microsoft Visual Studio, or you can add them manually.

Important

You will receive the error **System.UnauthorizedAccessException** when you attempt to use an API in the [Windows.Devices.PointOfService](#) namespace if you do not declare the **pointOfService** capability in your application manifest.

Declare capability using Manifest Designer

1. In **Solution Explorer**, expand the project node of your UWP application.
2. Double-click the **Package.appxmanifest** file.
If the manifest file is already open in the XML code view, Visual Studio prompts you to close the file.
3. Click the **Capabilities** tab
4. Click the checkbox next to **Point of Service** in the list of capabilities to enable the Point of Service device capability

Declare capability manually

1. In **Solution Explorer**, expand the project node of your UWP application.
2. Right-click the **Package.appxmanifest** file and choose **View Code**
3. Add the PointOfService DeviceCapability element to Capabilities section of your application manifest.

XML

```
<Capabilities>
  <DeviceCapability Name="pointOfService" />
</Capabilities>
```

Enumerating Point of Service devices

Article • 05/06/2023

This topic demonstrates how to [define a device selector](#) that is used to query devices available to the system and use this selector to enumerate Point of Service devices using one of the following methods:

Method 1: [Use a device picker](#)

Display a device picker UI and have the user choose a connected device. This method handles updating the list when devices are attached and removed, and is simpler and safer than other methods.

Method 2: [Get first available device](#)

Use [GetDefaultAsync](#) to access the first available device in a specific Point of Service device class.

Method 3: [Snapshot of devices](#)

Enumerate a snapshot of Point of Service devices that are present on the system at a given point in time. This is useful when you want to build your own UI or need to enumerate devices without displaying a UI to the user. [FindAllAsync](#) will hold back results until the entire enumeration is completed.

Method 4: [Enumerate and watch](#)

[DeviceWatcher](#) is a more powerful and flexible enumeration model that allows you to enumerate devices that are currently present, and also receive notifications when devices are added or removed from the system. This is useful when you want to maintain a current list of devices in the background for displaying in your UI rather than waiting for a snapshot to occur.

Define a device selector

A device selector enables you to limit the devices you are searching through when enumerating devices. This lets you get only relevant results and reduce the time it takes to enumerate the desired devices.

You can use the [GetDeviceSelector](#) method for the type of device that you're looking for and get the device selector for that type. For example, using [PosPrinter.GetDeviceSelector](#) will provide you with a selector to enumerate all [PosPrinters](#) attached to the system, including USB, network and Bluetooth POS printers.

Csharp

```
using Windows.Devices.PointOfService;

string selector = POSPrinter.GetDeviceSelector();
```

The **GetDeviceSelector** methods for the different device types are:

- [BarcodeScanner.GetDeviceSelector](#)
- [CashDrawer.GetDeviceSelector](#)
- [LineDisplay.GetDeviceSelector](#)
- [MagneticStripeReader.GetDeviceSelector](#)
- [PosPrinter.GetDeviceSelector](#)

Using a **GetDeviceSelector** method that takes a [PosConnectionTypes](#) value as a parameter, you can restrict your selector to enumerate local, network, or Bluetooth-attached POS devices, reducing the time it takes for the query to complete. The sample below shows a use of this method to define a selector that supports only locally attached POS printers.

Csharp

```
using Windows.Devices.PointOfService;

string selector = POSPrinter.GetDeviceSelector(PosConnectionTypes.Local);
```

Tip

See **Build a device selector** for building more advanced selector strings.

Method 1: Use a device picker

The [DevicePicker](#) class allows you to display a picker flyout that contains a list of devices for the user to choose from. You can use the [Filter](#) property to choose which types of devices to show in the picker. This property is of type [DevicePickerFilter](#). You can add device types to the filter using the [SupportedDeviceClasses](#) or [SupportedDeviceSelectors](#) property.

When you are ready to show the device picker, you can call the [PickSingleDeviceAsync](#) method, which will show the picker UI and return the selected device. You'll need to specify a [Rect](#) that will determine where the flyout appears. This method will return a [DeviceInformation](#) object, so to use it with the Point of Service APIs, you'll need to use

the **FromIdAsync** method for the particular device class that you want. You pass the [DeviceInformation.Id](#) property as the method's *deviceId* parameter, and get an instance of the device class as the return value.

The following code snippet creates a **DevicePicker**, adds a barcode scanner filter to it, has the user pick a device, and then creates a **BarcodeScanner** object based on the device ID:

C#

```
private async Task<BarcodeScanner> GetBarcodeScanner()
{
    DevicePicker devicePicker = new DevicePicker();

    devicePicker.Filter.SupportedDeviceSelectors.Add(BarcodeScanner.GetDeviceSelector());
    Rect rect = new Rect();
    DeviceInformation deviceInformation = await
devicePicker.PickSingleDeviceAsync(rect);
    BarcodeScanner barcodeScanner = await
BarcodeScanner.FromIdAsync(deviceInformation.Id);
    return barcodeScanner;
}
```

Method 2: Get first available device

The simplest way to get a Point of Service device is to use **GetDefaultAsync** to get the first available device within a Point of Service device class.

The sample below illustrates the use of [GetDefaultAsync](#) for [BarcodeScanner](#). The coding pattern is similar for all Point of Service device classes.

Csharp

```
using Windows.Devices.PointOfService;

BarcodeScanner barcodeScanner = await BarcodeScanner.GetDefaultAsync();
```

⊗ Caution

GetDefaultAsync must be used with care as it may return a different device from one session to the next. Many events can influence this enumeration resulting in a different first available device, including:

- Change in cameras attached to your computer

- Change in Point of Service devices attached to your computer
- Change in network-attached Point of Service devices available on your network
- Change in Bluetooth Point of Service devices within range of your computer
- Changes to the Point of Service configuration
- Installation of drivers or OPOS service objects
- Installation of Point of Service extensions
- Update to Windows operating system

Method 3: Snapshot of devices

In some scenarios you may want to build your own UI or need to enumerate devices without displaying a UI to the user. In these situations, you could enumerate a snapshot of devices that are currently connected or paired with the system using [DeviceInformation.FindAllAsync](#). This method will hold back any results until the entire enumeration is completed.

Tip

It is recommended to use the **GetDeviceSelector** method with the **PosConnectionTypes** parameter when using **FindAllAsync** to limit your query to the connection type desired. Network and Bluetooth connections can delay the results as their enumerations must complete before **FindAllAsync** results are returned.

Caution

FindAllAsync returns an array of devices. The order of this array can change from session to session, therefore it is not recommended to rely on a specific order by using a hardcoded index into the array. Use **DeviceInformation** properties to filter your results or provide a UI for the user to choose from.

This sample uses the selector defined above to take a snapshot of devices using **FindAllAsync** then enumerates through each of the items returned by the collection and writes the device name and ID to the debug output.

```
using Windows.Devices.Enumeration;

DeviceInformationCollection deviceCollection = await
DeviceInformation.FindAllAsync(selector);

foreach (DeviceInformation devInfo in deviceCollection)
{
    Debug.WriteLine("{0} {1}", devInfo.Name, devInfo.Id);
}
```

Tip

When working with the **Windows.Devices.Enumeration** APIs, you will frequently need to use **DeviceInformation** objects to obtain information about a specific device. For example, the **DeviceInformation.ID** property can be used to recover and reuse the same device if it is available in a future session and the **DeviceInformation.Name** property can be used for display purposes in your app. See the **DeviceInformation** reference page for information about additional properties available.

Method 4: Enumerate and watch

A more powerful and flexible method of enumerating devices is creating a **DeviceWatcher**. A device watcher enumerates devices dynamically, so that the application receives notifications if devices are added, removed, or changed after the initial enumeration is complete. A **DeviceWatcher** will allow you to detect when a network-connected device comes online, a Bluetooth device is in range, as well as if a locally connected device is unplugged so that you can take the appropriate action within your application.

This sample uses the selector defined above to create a **DeviceWatcher** as well as defines event handlers for the **Added**, **Removed**, and **Updated** notifications. You will need to fill in the details of the actions that you wish to take upon each notification.

Csharp

```
using Windows.Devices.Enumeration;

DeviceWatcher deviceWatcher = DeviceInformation.CreateWatcher(selector);
deviceWatcher.Added += DeviceWatcher_Added;
deviceWatcher.Removed += DeviceWatcher_Removed;
deviceWatcher.Updated += DeviceWatcher_Updated;
```

```
void DeviceWatcher_Added(DeviceWatcher sender, DeviceInformation args)
{
    // TODO: Add the DeviceInformation object to your collection
}

void DeviceWatcher_Removed(DeviceWatcher sender, DeviceInformationUpdate
args)
{
    // TODO: Remove the item in your collection associated with
DeviceInformationUpdate
}

void DeviceWatcher_Updated(DeviceWatcher sender, DeviceInformationUpdate
args)
{
    // TODO: Update your collection with information from
DeviceInformationUpdate
}
```

Tip

See [Enumerate and watch devices](#) for more details on the use of a `DeviceWatcher`.

See also

- [Getting started with Point of Service](#)
- [DeviceInformation Class](#)
- [PosPrinter Class](#)
- [PosConnectionTypes Enum](#)
- [BarcodeScanner Class](#)
- [DeviceWatcher Class](#)

Support and feedback

Find answers to your questions

Have questions? Ask us on either our [Docs Q&A forum with the UWP tag](#) or on [Stack Overflow with the pointofservice tag](#).

Help us locate your questions:

- Add the [pointofservice](#) tag to your question on Stack Overflow.
- Include the "UWP" term in your post on the Q&A forum

PointOfService device objects

Article • 05/06/2023

Learn how to create a [PointOfService device object](#) that will give you access to read-only properties of the peripheral and claim the peripheral for exclusive use in a Universal Windows Platform (UWP) application.

Creating a device object

Once you have identified the PointOfService device that you want to use, either from a fresh enumeration or a stored DeviceID, you just call [FromIdAsync](#) with the [DeviceID](#) that you have chosen programmatically or the user has selected to create a new Point of Service device object.

This sample attempts to create a new BarcodeScanner object with FromIdAsync using a DeviceID. If there is a failure creating the object a debug message is written.

Csharp

```
BarcodeScanner barcodeScanner = await
BarcodeScanner.FromIdAsync(DeviceId);

if(barcodeScanner != null)
{
    // after successful creation, claim the scanner for exclusive use
    and enable it to exchange data
}
else
{
    Debug.WriteLine("Failure to create barcodeScanner object");
}
```

Once you have a device object, you can then access the device's methods, properties and events.

Device object lifecycle

Before Windows 8, apps had a simple lifecycle. Win32 and .NET apps are either running or not running and PointOfService peripherals were usually claimed for the full app lifecycle. When a user minimizes them, or switches away from them, they continue to

run. This was fine until portable devices and power management became increasingly important.

Windows 8 introduced a new application model with UWP apps. At a high level, a new suspended state was added. A UWP app is suspended shortly after the user minimizes it or switches to another app. This means that the app's threads are stopped, the app is left in memory unless the operating system needs to reclaim resources, and any device objects representing PointOfService peripherals are automatically closed to allow other applications access to the peripherals. When the user switches back to the app, it can be quickly restored to a running state and restore PointOfService peripherals connections provided they are still available on resume.

You can detect when an object is closed for any reason with a `<DeviceObject>.Closed` event handler then make note of the device ID for re-establishing the connection in the future. Alternatively, you may wish to handle this on an App Suspend notification to save the device ID's for re-establishing the device connections on App Resume notification. Make sure that you do not double up on the event handlers and duplicate actions for the device object on both `<DeviceObject>.Closed` and App Suspend.

Tip

Please refer to the following topics for more information about Windows 10 Universal Windows Platform (UWP) application lifecycle:

- [Windows 10 Universal Windows Platform \(UWP\) app lifecycle](#)
- [Handle app suspension](#)
- [Handle app resume](#)

Point of Service device claim and enable model

Article • 05/06/2023

Use the Point of Service device claim and enable APIs to claim devices and enable them for I/O operations.

Claiming for exclusive use

After creating a [PointOfService device object](#), you must claim it using the appropriate claim method for the device type before you can use the device for input or output. Claim grants the application exclusive access to many of the device's functions to ensure that one application does not interfere with the use of the device by another application. Only one application at a time can claim a PointOfService device for exclusive use.

ⓘ Note

The claim action establishes an exclusive lock to a device, but does not put it into an operational state. See [Enable device for I/O operations](#) for more information.

APIs used to claim / release

Device	Claim	Release
BarcodeScanner	BarcodeScanner.ClaimScannerAsync	ClaimedBarcodeScanner.Close
CashDrawer	CashDrawer.ClaimDrawerAsync	ClaimedCashDrawer.Close
LineDisplay	LineDisplay.ClaimAsync	ClaimedLineDisplay.Close
MagneticStripeReader	MagneticStripeReader.ClaimReaderAsync	ClaimedMagneticStripeReader.Close
PosPrinter	PosPrinter.ClaimPrinterAsync	ClaimedPosPrinter.Close

Enable device for I/O operations

The claim action establishes exclusive rights to the device, but does not put it into an operational state. In order to receive events or perform any output operations you must

enable the device using **EnableAsync**. Conversely, you can call **DisableAsync** to stop listening to events from the device or stop performing output. You can also use **IsEnabled** to determine the state of your device.

APIs used enable / disable

Device	Enable	Disable	IsEnabled?
ClaimedBarcodeScanner	EnableAsync	DisableAsync	IsEnabled
ClaimedCashDrawer	EnableAsync	DisableAsync	IsEnabled
ClaimedLineDisplay	Not Applicable ¹	Not Applicable ¹	Not Applicable ¹
ClaimedMagneticStripeReader	EnableAsync	DisableAsync	IsEnabled
ClaimedPosPrinter	EnableAsync	DisableAsync	IsEnabled

¹ Line Display does not require you to explicitly enable the device for I/O operations. Enabling is performed automatically by the PointOfService LineDisplay APIs which perform I/O.

Code sample: claim and enable

This sample shows how to claim a barcode scanner device after you have successfully created a barcode scanner object.

Csharp

```
BarcodeScanner barcodeScanner = await
BarcodeScanner.FromIdAsync(DeviceId);

if(barcodeScanner != null)
{
    // after successful creation, claim the scanner for exclusive use
    claimedBarcodeScanner = await barcodeScanner.ClaimScannerAsync();

    if(claimedBarcodeScanner != null)
    {
        // after successful claim, enable scanner for data events to
fire
        await claimedBarcodeScanner.EnableAsync();
    }
    else
    {
        Debug.WriteLine("Failure to claim barcodeScanner");
    }
}
```

```
    }  
  }  
  else  
  {  
    Debug.WriteLine("Failure to create barcodeScanner object");  
  }  
}
```

Warning

A claim can be lost in the following circumstances:

1. Another app has requested a claim of the same device and your app did not issue a **RetainDevice** in response to the **ReleaseDeviceRequested** event. (See **Claim negotiation** below for more information.)
2. Your app has been suspended, which resulted in the device object being closed and as a result the claim is no longer valid. (See **Device object lifecycle** for more information.)

Claim negotiation

Since Windows is a multi-tasking environment, it is possible for multiple applications on the same computer to require access to peripherals in a cooperative manner. The PointOfService APIs provide a negotiation model that allows for multiple applications to share peripherals connected to the computer.

When a second application on the same computer requests a Claim for a PointOfService peripheral that is already claimed by another application, a **ReleaseDeviceRequested** event notification is published. The application with the active claim must respond to the event notification by calling **RetainDevice** if the application is currently using the device to avoid losing the claim.

If the application with the active claim does not respond with **RetainDevice** right away it is assumed that the application has been suspended or does not need the device and the claim is revoked and given to the new application.

The first step is to create an event handler that responds to the **ReleaseDeviceRequested** event with **RetainDevice**.

```

    /// <summary>
    /// Event handler for the ReleaseDeviceRequested event which occurs when
    /// the claimed barcode scanner receives a Claim request from another
    application
    /// </summary>
    void claimedBarcodeScanner_ReleaseDeviceRequested(object sender,
    ClaimedBarcodeScanner myScanner)
    {
        // Retain exclusive access to the device
        myScanner.RetainDevice();
    }

```

Next, register the event handler in association with your claimed device

Csharp

```

BarcodeScanner barcodeScanner = await
BarcodeScanner.FromIdAsync(DeviceId);

if(barcodeScanner != null)
{
    // after successful creation, claim the scanner for exclusive use
    claimedBarcodeScanner = await barcodeScanner.ClaimScannerAsync();

    if(claimedBarcodeScanner != null)
    {
        // register a release request handler to prevent loss of scanner
        during active use
        claimedBarcodeScanner.ReleaseDeviceRequested +=
        claimedBarcodeScanner_ReleaseDeviceRequested;

        // after successful claim, enable scanner for data events to
        fire
        await claimedBarcodeScanner.EnableAsync();
    }
    else
    {
        Debug.WriteLine("Failure to claim barcodeScanner");
    }
}
else
{
    Debug.WriteLine("Failure to create barcodeScanner object");
}

```

APIs used for claim negotiation

Claimed device

Release Notification

Retain Device

Claimed device	Release Notification	Retain Device
ClaimedBarcodeScanner	ReleaseDeviceRequested	RetainDevice
ClaimedCashDrawer	ReleaseDeviceRequested	RetainDevice
ClaimedLineDisplay	ReleaseDeviceRequested	RetainDevice
ClaimedMagneticStripeReader	ReleaseDeviceRequested	RetainDevice
ClaimedPosPrinter	ReleaseDeviceRequested	RetainDevice

PointOfService device sharing

Article • 05/06/2023

This topic shows how to share network or Bluetooth connected peripherals with other computers in an environment where multiple PCs rely on shared peripherals rather than dedicated peripherals attached to each computer.

Important APIs

- [BarcodeScanner.Dispose](#)
- [CashDrawer.Dispose](#)
- [LineDisplay.Dispose](#)
- [MagneticStripeReader.Dispose](#)
- [PosPrinter.Dispose](#)

Device sharing

Network and Bluetooth connected PointOfService peripherals are typically used in an environment where multiple client devices are sharing the same peripherals throughout the day. In a busy retail or food services environment any delay in the ability for a client device to attach to a peripheral has an impact on the efficiency in which an associate can close a transaction with the customer and move on to the next. In a quick service restaurant scenario where a receipt printer is used as a kitchen printer to transfer the details of a customer's order to the kitchen for preparation there will be multiple client devices taking orders from customers. Once the order is complete each client device should be able to claim the shared printer and immediately print the order for the kitchen.

In these environments, it is important for the application to fully **dispose** the device object so that another can claim the same device.

Disposing of a PosPrinter at the end of a 'using' block

Csharp

```
using Windows.Devices.PointOfService;
using(PosPrinter printer = await PosPrinter.FromIdAsync("Device ID"))
{
    if (printer != null)
    {
        // Exercise the printer.
    }
}
```

```
// When leaving this scope, printer.Dispose() is automatically invoked,  
// releasing the session we have with the printer.  
}
```

Disposing of a PosPrinter by calling Dispose() explicitly

C#

```
using Windows.Devices.PointOfService;  
  
PosPrinter printer = await PosPrinter.FromIdAsync("Device ID");  
if (printer != null)  
{  
    // Exercise the printer, then dispose of the printer explicitly.  
    printer.Dispose();  
}
```

Support and feedback

Find answers to your questions

Have questions? Ask us on either our [Docs Q&A forum with the UWP tag](#) or on [Stack Overflow with the pointofservice tag](#).

Help us locate your questions:

- Add the [pointofservice](#) tag to your question on Stack Overflow.
- Include the "UWP" term in your post on the Q&A forum

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Getting started with Point of Service

Article • 05/06/2023

Point of service, point of sale, or Point of Service devices are computer peripherals used to facilitate retail transactions. Examples of Point of Service devices include electronic cash registers, barcode scanners, magnetic stripe readers, and receipt printers.

Here you'll learn the basics of interfacing with Point of Service devices by using the Windows Runtime Point of Service APIs. We'll cover device enumeration, checking device capabilities, claiming devices, and device sharing. We use a barcode scanner device as an example, but almost all the guidance here applies to any UWP-compatible Point of Service device. (For a list of supported devices, see [Point of Service device support](#)).

Finding and connecting to Point of Service peripherals

Before a Point of Service device can be used by an app, it must be paired with the PC where the app is running. There are several ways to connect to Point of Service devices, either programmatically or through the Settings app.

Connecting to devices by using the Settings app

When you plug a Point of Service device like a barcode scanner into a PC, it shows up just like any other device. You can find it in the **Devices > Bluetooth & other devices** section of the Settings app. There you can pair with a Point of Service device by selecting **Add Bluetooth or other device**.

Some Point of Service devices may not appear in the Settings app until they are programmatically enumerated by using the Point of Service APIs.

Getting a single Point of Service device with `GetDefaultAsync`

In a simple use case, you may have just one Point of Service peripheral connected to the PC where the app is running and want to set it up as quickly as possible. To do that, retrieve the "default" device with the **GetDefaultAsync** method as shown here.

```
using Windows.Devices.PointOfService;

BarcodeScanner barcodeScanner = await BarcodeScanner.GetDefaultAsync();
```

If the default device is found, the device object retrieved is ready to be claimed. "Claiming" a device gives an application exclusive access to it, preventing conflicting commands from multiple processes.

ⓘ Note

If more than one Point of Service device is connected to the PC, **GetDefaultAsync** returns the first device it finds. For this reason, use **FindAllAsync** unless you're sure that only one Point of Service device is visible to the application.

Enumerating a collection of devices with FindAllAsync

When connected to more than one device, you must enumerate the collection of **PointOfService** device objects to find the one you want to claim. For example, the following code creates a collection of all the barcode scanners currently connected, and then searches the collection for a scanner with a specific name.

Csharp

```
using Windows.Devices.Enumeration;
using Windows.Devices.PointOfService;

string selector = BarcodeScanner.GetDeviceSelector();
DeviceInformationCollection deviceCollection = await
DeviceInformation.FindAllAsync(selector);

foreach (DeviceInformation devInfo in deviceCollection)
{
    Debug.WriteLine("{0} {1}", devInfo.Name, devInfo.Id);
    if (devInfo.Name.Contains("1200G"))
    {
        Debug.WriteLine(" Found one");
    }
}
```

Scoping the device selection

When connecting to a device, you may want to limit your search to a subset of Point of Service peripherals that your app has access to. Using the **GetDeviceSelector** method,

you can scope the selection to retrieve devices connected only by a certain method (Bluetooth, USB, etc.). You can create a selector that searches for devices over **Bluetooth**, **IP**, **Local**, or **All connection types**. This can be useful, as wireless device discovery takes a long time compared to local (wired) discovery. You can ensure a deterministic wait time for local device connection by limiting **FindAllAsync** to **Local** connection types. For example, this code retrieves all barcode scanners accessible via a local connection.

Csharp

```
string selector =  
BarcodeScanner.GetDeviceSelector(PosConnectionTypes.Local);  
DeviceInformationCollection deviceCollection = await  
DeviceInformation.FindAllAsync(selector);
```

Reacting to device connection changes with DeviceWatcher

As your app runs, sometimes devices will be disconnected or updated, or new devices will need to be added. You can use the **DeviceWatcher** class to access device-related events, so your app can respond accordingly. Here's example of how to use **DeviceWatcher**, with method stubs to be called if a device is added, removed, or updated.

Csharp

```
DeviceWatcher deviceWatcher = DeviceInformation.CreateWatcher(selector);  
deviceWatcher.Added += DeviceWatcher_Added;  
deviceWatcher.Removed += DeviceWatcher_Removed;  
deviceWatcher.Updated += DeviceWatcher_Updated;  
  
void DeviceWatcher_Added(DeviceWatcher sender, DeviceInformation args)  
{  
    // TODO: Add the DeviceInformation object to your collection  
}  
  
void DeviceWatcher_Removed(DeviceWatcher sender, DeviceInformationUpdate  
args)  
{  
    // TODO: Remove the item in your collection associated with  
    DeviceInformationUpdate  
}  
  
void DeviceWatcher_Updated(DeviceWatcher sender, DeviceInformationUpdate  
args)  
{  
    // TODO: Update your collection with information from
```

```
DeviceInformationUpdate
}
```

Checking the capabilities of a Point of Service device

Even within a device class, such as barcode scanners, the attributes of each device may vary considerably between models. If your app requires a specific device attribute, you may need to inspect each connected device object to determine whether the attribute is supported. For example, perhaps your business requires that labels be created using a specific barcode printing pattern. Here's how you could check to see whether a connected barcode scanner supports a symbology.

⚠ Note

A symbology is the language mapping that a barcode uses to encode messages.

Csharp

```
try
{
    BarcodeScanner barcodeScanner = await
BarcodeScanner.FromIdAsync(deviceId);
    if (await
barcodeScanner.IsSymbologySupportedAsync(BarcodeSymbologies.Code32))
    {
        Debug.WriteLine("Has symbology");
    }
}
catch (Exception ex)
{
    Debug.WriteLine("FromIdAsync() - " - ex.Message);
}
```

Using the Device.Capabilities class

The **Device.Capabilities** class is an attribute of all Point of Service device classes and can be used to get general information about each device. For example, this example determines whether a device supports statistics reporting and, if it does, retrieves statistics for any types supported.

Csharp

```

try
{
    if (barcodeScanner.Capabilities.IsStatisticsReportingSupported)
    {
        Debug.WriteLine("Statistics reporting is supported");

        string[] statTypes = new string[] {""};
        IBuffer ibuffer = await
barcodeScanner.RetrieveStatisticsAsync(statTypes);
    }
}
catch (Exception ex)
{
    Debug.WriteLine("EX: RetrieveStatisticsAsync() - " - ex.Message);
}

```

Claiming a Point of Service device

Before you can use a Point of Service device for active input or output, you must claim it, granting the application exclusive access to many of its functions. This code shows how to claim a barcode scanner device, after you've found the device by using one of the methods described earlier.

Csharp

```

try
{
    claimedBarcodeScanner = await barcodeScanner.ClaimScannerAsync();
}
catch (Exception ex)
{
    Debug.WriteLine("EX: ClaimScannerAsync() - " - ex.Message);
}

```

Retaining the device

When using a Point of Service device over a network or Bluetooth connection, you may wish to share the device with other apps on the network. (For more info about this, see [Sharing Devices](#).) In other cases, you may want to hold on to the device for prolonged use. This example shows how to retain a claimed barcode scanner after another app has requested that the device be released.

Csharp

```

claimedBarcodeScanner.ReleaseDeviceRequested +=
claimedBarcodeScanner_ReleaseDeviceRequested;

void claimedBarcodeScanner_ReleaseDeviceRequested(object sender,
ClaimedBarcodeScanner e)
{
    e.RetainDevice(); // Retain exclusive access to the device
}

```

Input and output

After you've claimed a device, you're almost ready to use it. To receive input from the device, you must set up and enable a delegate to receive data. In the example below, we claim a barcode scanner device, set its decode property, and then call **EnableAsync** to enable decoded input from the device. This process varies between device classes, so for guidance about how to set up a delegate for non-barcode devices, refer to the relevant [UWP app sample](#).

Csharp

```

try
{
    claimedBarcodeScanner = await barcodeScanner.ClaimScannerAsync();
    if (claimedBarcodeScanner != null)
    {
        claimedBarcodeScanner.DataReceived +=
claimedBarcodeScanner_DataReceived;
        claimedBarcodeScanner.IsDecodeDataEnabled = true;
        await claimedBarcodeScanner.EnableAsync();
    }
}
catch (Exception ex)
{
    Debug.WriteLine("EX: ClaimScannerAsync() - " - ex.Message);
}

void claimedBarcodeScanner_DataReceived(ClaimedBarcodeScanner sender,
BarcodeScannerDataReceivedEventArgs args)
{
    string symbologyName =
BarcodeSymbolologies.GetName(args.Report.ScanDataType);
    var scanDataLabelReader =
DataReader.FromBuffer(args.Report.ScanDataLabel);
    string barcode =
scanDataLabelReader.ReadString(args.Report.ScanDataLabel.Length);
}

```

Sharing a device between apps

Point of Service devices are often used in cases where more than one app will need to access them in a brief period. A device can be shared when connected to multiple apps locally (USB or other wired connection), or through a Bluetooth or IP network.

Depending on the needs of each app, one process may need to dispose of its claim on the device. This code disposes of our claimed barcode scanner device, allowing other apps to claim and use it.

Csharp

```
if (claimedBarcodeScanner != null)
{
    claimedBarcodeScanner.Dispose();
    claimedBarcodeScanner = null;
}
```

ⓘ Note

Both the claimed and unclaimed Point of Service device classes implement the **IClosable interface**. If a device is connected to an app via network or Bluetooth, both the claimed and unclaimed objects must be disposed of before another app can connect.

See also

- [Barcode scanner sample](#) ↗
- [Cash drawer sample](#) ↗
- [Line display sample](#) ↗
- [Magnetic stripe reader sample](#) ↗
- [POSPrinter sample](#) ↗

Supported Point of Service peripherals

Article • 05/04/2023

This topic describes the hardware support for each of the Point of Service device classes.

Barcode Scanner

Connectivity	Support
USB	Windows contains an in-box class driver for USB connected barcode scanners which is based on the HID POS Scanner Usage Table (8c) specification defined by USB.org . See the table below for a list of known compatible devices. Consult the manual for your barcode scanner or contact the manufacturer to determine how to configure your scanner in USB.HID.POS Scanner mode. Windows also supports implementation of vendor specific drivers to support additional barcode scanners that do not support the USB.HID.POS Scanner standard. Please check with your barcode scanner manufacturer for vendor specific driver availability. Barcode scanner manufacturers please consult the Barcode Scanner Driver Design Guide for information on creating a custom barcode scanner driver
Bluetooth	Windows supports Serial Port Protocol - Simple Serial Interface (SPP-SSI) based Bluetooth barcode scanners. See the table below for a list of known compatible devices. Consult the manual for your barcode scanner or contact the manufacturer to determine how to configure your scanner in SPP-SSI mode.
Webcam	Starting with Windows 10 version 1803, you can read barcodes through a standard camera lens from a Universal Windows Application. It is recommended that you use a camera that supports Auto Focus and a minimum resolution of 1920 x 1440. Some lower resolution cameras can read standard barcodes if the barcode is printed large enough. Barcodes with thinner elements may require higher resolution cameras.

Manufacturer	Model	Capability	Connection	Type	Mode
Code	Reader™ 950	2D	USB	Handheld	HID POS Scanner
Code	Reader™ 1021	2D	USB	Handheld	HID POS Scanner
Code	Reader™ 1421	2D	USB	Handheld	HID POS Scanner

Manufacturer	Model	Capability	Connection	Type	Mode
Code	Reader™ 5000	2D	USB	Presentation	HID POS Scanner
Honeywell	Genesis 7580g	2D	USB	Presentation	HID POS Scanner
Honeywell	Granit 198Xi	2D	USB	Handheld	HID POS Scanner
Honeywell	Granit 191Xi	2D	USB	Handheld	HID POS Scanner
Honeywell	N5680	2D	Internal	Component	HID POS Scanner
Honeywell	N3680	2D	Internal	Component	HID POS Scanner
Honeywell	Orbit 7190g	2D	USB	Presentation	HID POS Scanner
Honeywell	Stratos 2700	2D	USB	In Counter	HID POS Scanner
Honeywell	Voyager 1200g	1D	USB	Handheld	HID POS Scanner
Honeywell	Voyager 1202g	1D	USB	Handheld	HID POS Scanner
Honeywell	Voyager 1202-bf	1D	USB	Handheld	HID POS Scanner
Honeywell	Voyager 145Xg	1D / 2D ¹	USB	Handheld	HID POS Scanner
Honeywell	Voyager 1602g	2D	USB	Handheld	HID POS Scanner
Honeywell	Xenon 1900g	2D	USB	Handheld	HID POS Scanner
Honeywell	Xenon 1902g	2D	USB	Handheld	HID POS Scanner
Honeywell	Xenon 1902g-bf	2D	USB	Handheld	HID POS Scanner
Honeywell	Xenon 1900h	2D	USB	Handheld	HID POS Scanner

Manufacturer	Model	Capability	Connection	Type	Mode
Honeywell	Xenon 1902h	2D	USB	Handheld	HID POS Scanner
HP	Value Barcode Scanner (HR2150)	2D	USB	Handheld	HID POS Scanner
Intermec	SG20	2D	USB	Handheld	HID POS Scanner
Socket Mobile	CHS 7Ci	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	CHS 7Di	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	CHS 7Mi	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	CHS 7Pi	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	CHS 8Ci	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	DuraScan D700	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	DuraScan D730	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	DuraScan D740	2D	Bluetooth	Handheld	Serial Port Profile (SPP)

Manufacturer	Model	Capability	Connection	Type	Mode
Socket Mobile	SocketScan S700	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	SocketScan S730	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	SocketScan S740	2D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	SocketScan S800	1D	Bluetooth	Handheld	Serial Port Profile (SPP)
Socket Mobile	SocketScan S850	2D	Bluetooth	Handheld	Serial Port Profile (SPP)
Zebra	DS2208 ²	2D	USB	Handheld	HID POS Scanner
Zebra	DS2278	2D	USB	Handheld	HID POS Scanner
Zebra	DS8108 ³	2D	USB	Handheld	HID POS Scanner
Zebra	DS8178 ⁴	2D	USB	Handheld	HID POS Scanner

¹ Upgradable to support 2D barcodes through Honeywell

² Minimum firmware 009 (2018.07.09) required. Upgradable using Zebra [123Scan](#).

³ Minimum firmware 016 (2018.01.18) required. Upgradable using Zebra [123Scan](#).

⁴ Minimum firmware 023 (2019.03.11) required. Upgradable using Zebra [123Scan](#).

Windows devices with built-in barcode scanner

Manufacturer	Model	Operating System
--------------	-------	------------------

Manufacturer	Model	Operating System
Innowi	ChecOut-M	Windows 10

Windows Mobile devices with built-in barcode scanner

Manufacturer	Model	Operating System
Bluebird	EF400	Windows Mobile
Bluebird	EF500	Windows Mobile
Bluebird	EF500R	Windows Mobile
Honeywell	CT50	Windows Mobile
Honeywell	D75e	Windows Mobile
Janam	XT2	Windows Mobile
Panasonic	FZ-E1	Windows Mobile
Panasonic	FZ-F1	Windows Mobile
PointMobile	PM80	Windows Mobile
Zebra	TC700j	Windows Mobile
HP	Elite X3 Jacket	Windows Mobile

Cash Drawer

Connectivity	Support
Network/Bluetooth	Connection directly to the cash drawer can be made over the network or through Bluetooth, depending on the capabilities of the cash drawer unit. APG Cash Drawer: NetPRO, BluePRO
DK port	Cash drawers that do not have network or Bluetooth capabilities can be connected via the DK port on a supported Receipt Printer or the Star Micronics DK-AirCash accessory.
OPOS	Supports any OPOS compatible Cash Drawers via OPOS service objects provided by the manufacturer. Install the OPOS drivers as per the device manufacturers installation instructions.

Customer Display (LineDisplay)

Supports any OPOS compatible line displays via OPOS service objects provided by the manufacturer. Install the OPOS drivers as per the device manufacturers installation instructions.

Magnetic Stripe Reader

Windows provides support for the following magnetic stripe readers from Magtek and IDTech based on their Vendor ID and Product ID (VID/PID).

Manufacturer	Model(s)	Part Number
IDTech	SecureMag (VID:0ACD PID:2010)	IDRE-3x5xxxx
	MiniMag (VID:0ACD PID:0500)	IDMB-3x5xxxx
Magtek	MagneSafe (VID:0801 PID:0011)	210730xx
	Dynamag (VID:0801 PID:0002)	210401xx

Windows supports implementation of additional vendor specific drivers to support additional magnetic stripe readers. Please check with your magnetic stripe reader manufacturer for availability. Magnetic stripe reader manufacturers please consult the [Magnetic Stripe Reader Driver Design Guide](#) for information on creating a custom magnetic stripe reader driver.

Receipt Printer (POSPrinter)

Connectivity	Support
Network and Bluetooth	Windows supports network and Bluetooth connected receipt printers using the Epson ESC/POS printer control language. The printers listed below are discovered automatically using POSPrinter APIs. Additional receipt printers which provide an ESC/POS emulation may also work but would need to be associated using an out of band pairing process. Note: slip station and journal stations are not supported through this method.
OPOS	Supports any OPOS compatible receipt printers via OPOS service objects. Install the OPOS drivers as per the device manufacturers installation instructions.

Stationary Receipt Printers (Network/Bluetooth)

Manufacturer	Model(s)
Epson	TM-T88V, TM-T70, TM-T20, TM-U220

Mobile Receipt Printers (Bluetooth)

Manufacturer	Model(s)
Epson	Mobilink P20 (TM-P20), Mobilink P60 (TM-P60), Mobilink P80 (TM-P80)

Barcode scanner

Article • 05/04/2023

This section provides guidance for creating Universal Windows Platform (UWP) apps that use a barcode scanner.

In this section

Topic	Description
Configure a barcode scanner	Learn how to configure a barcode scanner for the intended application.
Use a software trigger	Learn how to control the act of scanning from software.
Working with symbologies	Learn how to determine the barcode types a barcode scanner supports and control which barcode types are recognized by the barcode scanner from your application.
Obtain and understand barcode data	Learn how to obtain and interpret the barcode data that you scan.
Camera Barcode Scanner	Read barcodes through a standard camera lens from a Universal Windows Platform application. Requires Windows 10 Version 1803 or later.

Configure a barcode scanner

Article • 05/06/2023

This topic describes how barcode scanners can be configured in Universal Windows Platform (UWP) apps.

Keyboard wedge mode

Many barcode scanners support **keyboard wedge** mode, which makes the barcode scanner appear as a keyboard to Windows. This enables scanning barcodes into applications that are not barcode scanner-aware such as Notepad. When you scan a barcode in this mode, the decoded data from the barcode scanner gets inserted at your insertion point as if you typed the data using your keyboard. If you want more control over your barcode scanner from your UWP application, you will need to configure it in a non-keyboard wedge mode.

USB barcode scanner

A USB connected barcode scanner must be configured in **HID POS Scanner** mode to work with the barcode scanner driver that is included in Windows. This driver is an implementation of the **HID Point of Sale Usage Tables** specification published to [USB-HID](#). Please refer to your barcode scanner documentation or contact your barcode scanner manufacturer for instructions to enable the **HID POS Scanner** mode. Once configured as a **HID POS Scanner** your barcode scanner will appear in Device Manager under the **POS Barcode Scanner** node as **POS HID Barcode scanner**.

Your barcode scanner manufacturer may also have a vendor specific driver that supports the UWP Barcode Scanner APIs using a mode other than **HID POS Scanner**. If you have already installed a manufacturer-provided driver compatible with UWP Barcode Scanner APIs, you may see a vendor-specific device listed under **POS Barcode Scanner** in Device Manager.

Bluetooth barcode scanner

A Bluetooth connected scanner must be configured in **Serial Port Protocol - Simple Serial Interface (SPP-SSI)** mode to work with the UWP Barcode Scanner APIs. Please refer to your barcode scanner documentation or contact your barcode scanner manufacturer for instructions to enable the **SPP-SSI mode**.

Before you can use your Bluetooth barcode scanner you must pair it using **Settings > Devices > Bluetooth & other devices > Add Bluetooth or other device**.

You can initiate and control the pairing process using the [Windows.Devices.Enumeration](#) namespace. See [Pair Devices](#) for more information.

Support and feedback

Find answers to your questions

Have questions? Ask us on either our [Docs Q&A forum with the UWP tag](#) or on [Stack Overflow with the pointofservice tag](#).

Help us locate your questions:

- Add the [pointofservice](#) tag to your question on Stack Overflow.
- Include the "UWP" term in your post on the Q&A forum

Use a software trigger with a barcode scanner

Article • 05/06/2023

This topic explains how to control the barcode scanning process programmatically using an asynchronous software trigger.

It can be useful to control the act of scanning from software if you are using a barcode scanner in presentation mode or if the scanner does not have a physical trigger such as a camera-based barcode scanner. You can initiate the scan process by calling [StartSoftwareTriggerAsync](#).

Depending on the value of [IsDisabledOnDataReceived](#) the scanner may scan only one barcode then stop or scan continuously until you call [StopSoftwareTriggerAsync](#).

Set the desired value of [IsDisabledOnDataReceived](#) to control the scanner behavior when a barcode is decoded.

Value	Description
True	Scan only one barcode then stop
False	Continuously scan barcodes without stopping

Important

Confirm that your barcode scanner supports the use of software trigger by first checking the property [IsSoftwareTriggerSupported](#).

The following example shows how to initiate scanning using a software trigger, which will stop scanning once it scans one barcode:

C#

```
private void SoftwareTrigger(BarcodeScanner barcodeScanner,
    ClaimedBarcodeScanner claimedBarcodeScanner)
{
    if (barcodeScanner.Capabilities.IsSoftwareTriggerSupported)
    {
        claimedBarcodeScanner.IsDisabledOnDataReceived = true;
        await claimedBarcodeScanner.StartSoftwareTriggerAsync();
    }
}
```


Support and feedback

Find answers to your questions

Have questions? Ask us on either our [Docs Q&A forum with the UWP tag](#) or on [Stack Overflow with the pointofservice tag](#).

Help us locate your questions:

- Add the [pointofservice](#) tag to your question on Stack Overflow.
- Include the ["UWP"](#) term in your post on the Q&A forum

Working with barcode scanner symbologies

Article • 05/06/2023

This topic explains how to use the Universal Windows Platform (UWP) barcode scanner APIs to process barcode symbologies without manually configuring the scanner.

A [barcode symbology](#) is the mapping of data to a specific barcode format. Some common symbologies include UPC, Code 128, QR Code, and so on.

Determine which symbologies are supported

Since your application may be used with different barcode scanner models from multiple manufacturers, you may want to query the scanner to determine the list of symbologies that it supports. This can be useful if your application requires a specific symbology that may not be supported by all scanners or you need to enable symbologies that have been either manually or programmatically disabled on the scanner.

Once you have a [BarcodeScanner](#) object by using [BarcodeScanner.FromIdAsync](#), call [GetSupportedSymbologiesAsync](#) to obtain a list of symbologies supported by the device.

The following example gets a list of the supported symbologies of the barcode scanner, and displays them in a text block:

C#

```
private void DisplaySupportedSymbologies(BarcodeScanner barcodeScanner,
    TextBlock textBlock)
{
    var supportedSymbologies = await
        barcodeScanner.GetSupportedSymbologiesAsync();

    foreach (uint item in supportedSymbologies)
    {
        string symbology = BarcodeSymbologies.GetName(item);
        textBlock.Text += (symbology + "\n");
    }
}
```

Determine if a specific symbology is supported

To determine if the scanner supports a specific symbology you can call [IsSymbologySupportedAsync](#).

The following example checks if the barcode scanner supports the **Code32** symbology:

C#

```
bool symbologySupported = await  
barcodeScanner.IsSymbologySupportedAsync(BarcodeSymbologies.Code32);
```

Change which symbologies are recognized

In some cases, you may want to use a subset of symbologies that the barcode scanner supports. This is particularly useful to block symbologies that you do not intend to use in your application. For example, to ensure a user scans the right barcode, you could constrain scanning to UPC or EAN when acquiring item SKUs and constrain scanning to Code 128 when acquiring serial numbers.

Once you know the symbologies that your scanner supports, you can set the symbologies that you want it to recognize. This can be done after you have established a [ClaimedBarcodeScanner](#) object using [ClaimScannerAsync](#). You can call [SetActiveSymbologiesAsync](#) to enable a specific set of symbologies while those omitted from your list are disabled.

The following example sets the active symbologies of a claimed barcode scanner to [Code39](#) and [Code39Ex](#):

C#

```
private async void SetSymbologies(ClaimedBarcodeScanner  
claimedBarcodeScanner)  
{  
    var symbologies = new List<uint>{ BarcodeSymbologies.Code39,  
BarcodeSymbologies.Code39Ex };  
    await claimedBarcodeScanner.SetActiveSymbologiesAsync(symbologies);  
}
```

Barcode symbology attributes

Different barcode symbologies can have different attributes, such as supporting multiple decode lengths, transmitting the check digit to the host as part of the raw data, and check digit validation. With the [BarcodeSymbologyAttributes](#) class, you can get and set these attributes for a given [ClaimedBarcodeScanner](#) and barcode symbology.

You can get the attributes of a given symbology with [GetSymbologyAttributesAsync](#). The following code snippet gets the attributes of the Upca symbology for a **ClaimedBarcodeScanner**.

C#

```
BarcodeSymbologyAttributes barcodeSymbologyAttributes =  
    await  
claimedBarcodeScanner.GetSymbologyAttributesAsync(BarcodeSymbologies.Upca);
```

When you've finished modifying the attributes and are ready to set them, you can call [SetSymbologyAttributesAsync](#). This method returns a **bool**, which is **true** if the attributes were successfully set.

C#

```
bool success = await claimedBarcodeScanner.SetSymbologyAttributesAsync(  
    BarcodeSymbologies.Upca, barcodeSymbologyAttributes);
```

Restrict scan data by data length

Some symbologies are variable length such as Code 39 or Code 128. Barcodes of these symbologies can be located near each other containing different data often of specific length. Setting the specific length of the data you require can prevent invalid scans.

Before setting the decode length, check whether the barcode symbology supports multiple lengths with [IsDecodeLengthSupported](#). Once you know that it's supported, you can set the [DecodeLengthKind](#), which is of type [BarcodeSymbologyDecodeLengthKind](#). This property can be any of the following values:

- **AnyLength**: Decode lengths of any number.
- **Discrete**: Decode lengths of either [DecodeLength1](#) or [DecodeLength2](#) single-byte characters.
- **Range**: Decode lengths between **DecodeLength1** and **DecodeLength2** single-byte characters. The order of **DecodeLength1** and **DecodeLength2** do not matter (either can be higher or lower than the other).

Finally, you can set the values of **DecodeLength1** and **DecodeLength2** to control the length of the data you require.

The following code snippet demonstrates setting the decode length:

C#

```

private async Task<bool> SetDecodeLength(
    ClaimedBarcodeScanner scanner,
    uint symbology,
    BarcodeSymbologyDecodeLengthKind kind,
    uint decodeLength1,
    uint decodeLength2)
{
    bool success = false;
    BarcodeSymbologyAttributes attributes = await
scanner.GetSymbologyAttributesAsync(symbology);

    if (attributes.IsDecodeLengthSupported)
    {
        attributes.DecodeLengthKind = kind;
        attributes.DecodeLength1 = decodeLength1;
        attributes.DecodeLength2 = decodeLength2;
        success = await scanner.SetSymbologyAttributesAsync(symbology,
attributes);
    }

    return success;
}

```

Check digit transmission

Another attribute you can set on a symbology is whether the check digit will be transmitted to the host as part of the raw data. Before setting this, make sure that the symbology supports check digit transmission with [IsCheckDigitTransmissionSupported](#). Then, set whether check digit transmission is enabled with [IsCheckDigitTransmissionEnabled](#).

The following code snippet demonstrates setting check digit transmission:

C#

```

private async Task<bool> SetCheckDigitTransmission(ClaimedBarcodeScanner
scanner, uint symbology, bool isEnabled)
{
    bool success = false;
    BarcodeSymbologyAttributes attributes = await
scanner.GetSymbologyAttributesAsync(symbology);

    if (attributes.IsCheckDigitTransmissionSupported)
    {
        attributes.IsCheckDigitTransmissionEnabled = isEnabled;
        success = await scanner.SetSymbologyAttributesAsync(symbology,
attributes);
    }
}

```

```
        return success;
    }
```

Check digit validation

You can also set whether the barcode check digit will be validated. Before setting this, make sure that the symbology supports check digit validation with [IsCheckDigitValidationSupported](#). Then, set whether check digit validation is enabled with [IsCheckDigitValidationEnabled](#).

The following code snippet demonstrates setting check digit validation:

C#

```
private async Task<bool> SetCheckDigitValidation(ClaimedBarcodeScanner
scanner, uint symbology, bool isEnabled)
{
    bool success = false;
    BarcodeSymbologyAttributes attributes = await
scanner.GetSymbologyAttributesAsync(symbology);

    if (attributes.IsCheckDigitValidationSupported)
    {
        attributes.IsCheckDigitValidationEnabled = isEnabled;
        success = await scanner.SetSymbologyAttributesAsync(symbology,
attributes);
    }

    return success;
}
```

Support and feedback

Find answers to your questions

Have questions? Ask us on either our [Docs Q&A forum with the UWP tag](#) or on [Stack Overflow with the pointofservice tag](#).

Help us locate your questions:

- Add the [pointofservice](#) tag to your question on Stack Overflow.
- Include the "UWP" term in your post on the Q&A forum

See also

- [Barcode scanner](#)
- [BarcodeSymbologies Class](#)
- [BarcodeScanner Class](#)
- [ClaimedBarcodeScanner Class](#)
- [BarcodeSymbologyAttributes Class](#)
- [BarcodeSymbologyDecodeLengthKind Enum](#)

Obtain and understand barcode data

Article • 05/06/2023

This topic describes how to obtain data from a barcode scanner in a [BarcodeScannerReport](#) object and understand its format and contents.

Once you've set up your barcode scanner and scan a barcode, the [DataReceived](#) event is raised. The [ClaimedBarcodeScanner](#) should subscribe to this event. The **DataReceived** event passes a [BarcodeScannerDataReceivedEventArgs](#) object, which you can use to access the barcode data.

Subscribe to the DataReceived event

Once you have a **ClaimedBarcodeScanner**, use it to subscribe to the **DataReceived** event:

C#

```
claimedBarcodeScanner.DataReceived += ClaimedBarcodeScanner_DataReceived;
```

The event handler will be passed the **ClaimedBarcodeScanner** and a **BarcodeScannerDataReceivedEventArgs** object. You can access the barcode data through this object's [Report](#) property, which is of type [BarcodeScannerReport](#).

C#

```
private async void ClaimedBarcodeScanner_DataReceived(ClaimedBarcodeScanner sender, BarcodeScannerDataReceivedEventArgs args)
{
    // Parse the data
}
```

Get the data

Once you have the **BarcodeScannerReport**, you can access and parse the barcode data. **BarcodeScannerReport** has three properties:

- [ScanData](#): The full, raw barcode data.
- [ScanDataLabel](#): The decoded barcode label, which does not include the header, checksum, and other miscellaneous information.

- [ScanDataType](#): The decoded barcode label type. Possible values are defined in the [BarcodeSymbologies](#) class.

If you want to access either **ScanDataLabel** or **ScanDataType**, you must first set [IsDecodeDataEnabled](#) to **true**.

C#

```
claimedBarcodeScanner.IsDecodeDataEnabled = true;
```

Get the scan data type

Getting the decoded barcode label type is fairly trivial—we simply call [GetName](#) on **ScanDataType**.

C#

```
private string GetSymbology(BarcodeScannerDataReceivedEventArgs args)
{
    return BarcodeSymbologies.GetName(args.Report.ScanDataType);
}
```

Get the scan data label

To get the decoded barcode label, there are a few things you have to be aware of. Only certain data types contain encoded text, so you should first check if the symbology can be converted to a string, and then convert the buffer we get from **ScanDataLabel** to an encoded UTF-8 string.

C#

```
private string GetDataLabel(BarcodeScannerDataReceivedEventArgs args)
{
    uint scanDataType = args.Report.ScanDataType;

    // Only certain data types contain encoded text.
    // To keep this simple, we'll just decode a few of them.
    if (args.Report.ScanDataLabel == null)
    {
        return "No data";
    }

    // This is not an exhaustive list of symbologies that can be converted
    to a string.
    else if (scanDataType == BarcodeSymbologies.UPCA ||
        scanDataType == BarcodeSymbologies.UPCAAdd2 ||
```

```

        scanDataType == BarcodeSymbologies.UPCAAdd5 ||
        scanDataType == BarcodeSymbologies.UPCA ||
        scanDataType == BarcodeSymbologies.UPCAAdd2 ||
        scanDataType == BarcodeSymbologies.UPCAAdd5 ||
        scanDataType == BarcodeSymbologies.EAN8 ||
        scanDataType == BarcodeSymbologies.TFStd)
    {
        // The UPC, EAN8, and 2 of 5 families encode the digits 0..9
        // which are then sent to the app in a UTF8 string (like "01234").
        return
        CryptographicBuffer.ConvertBinaryToString(BinaryStringEncoding.Utf8,
        args.Report.ScanDataLabel);
    }

    // Some other symbologies (typically 2-D symbologies) contain binary
    data that
    // should not be converted to text.
    else
    {
        return "Decoded data unavailable.";
    }
}

```

Get the raw scan data

To get the full, raw data from the barcode, we simply convert the buffer we get from **ScanData** into a string.

C#

```

private string GetRawData(BarcodeScannerDataReceivedEventArgs args)
{
    // Get the full, raw barcode data.
    if (args.Report.ScanData == null)
    {
        return "No data";
    }

    // Just to show that we have the raw data, we'll print the value of the
    bytes.
    else
    {
        return
        CryptographicBuffer.ConvertBinaryToString(BinaryStringEncoding.Utf8,
        args.Report.ScanData);
    }
}

```

This data is typically in the format delivered from the scanner. Message header and trailer information are removed, however, since they do not contain useful information

for an application and are likely to be scanner-specific.

Common header information is a prefix character (such as an STX character). Common trailer information is a terminator character (such as an ETX or CR character) and a block check character if one is generated by the scanner.

This property should include a symbology character if one is returned by the scanner (for example, an **A** for UPC-A). It should also include check digits if they are present in the label and returned by the scanner. (Note that both symbology characters and check digits may or may not be present, depending upon the scanner configuration. The scanner will return them if present, but will not generate or calculate them if they are absent.)

Some merchandise may be marked with a supplemental barcode. This barcode is typically placed to the right of the main barcode, and consists of an additional two or five characters of information. If the scanner reads merchandise that contains both main and supplemental barcodes, the supplemental characters are appended to the main characters, and the result is delivered to the application as one label. (Note that a scanner may support a configuration that enables or disables the reading of supplemental codes.)

Some merchandise may be marked with multiple labels, sometimes called *multisymbol labels* or *tiered labels*. These barcodes are typically arranged vertically, and may be of the same or different symbology. If the scanner reads merchandise that contains multiple labels, each barcode is delivered to the application as a separate label. This is necessary due to the current lack of standardization of these barcode types. One is not able to determine all variations based upon the individual barcode data. Therefore, the application will need to determine when a multiple label barcode has been read based upon the data returned. (Note that a scanner may or may not support reading of multiple labels.)

This value is set prior to a **DataReceived** event being raised to the application.

Support and feedback

Find answers to your questions

Have questions? Ask us on either our [Docs Q&A forum with the UWP tag](#) or on [Stack Overflow with the pointofservice tag](#).

Help us locate your questions:

- Add the [pointofservice](#) tag to your question on Stack Overflow.
- Include the ["UWP"](#) term in your post on the Q&A forum

See also

- [Barcode scanner](#)
- [ClaimedBarcodeScanner Class](#)
- [BarcodeScannerDataReceivedEventArgs Class](#)
- [BarcodeScannerReport Class](#)
- [BarcodeSymbologies Class](#)

Camera barcode scanner

Article • 01/09/2024

Requires Windows 10 Version 1803 or later.

A camera barcode scanner consists of a standard lens camera with auto-focus capabilities (attached to a computer) combined with a software decoder, which Windows dynamically pairs to create a fully functional [barcode scanner](#) for Universal Windows Platform (UWP) apps.

ⓘ Note

The software decoder built into Windows 10/11 is provided by [Digimarc Corporation](#) [↗].

In this section

 Expand table

Title	Description
System requirements	List of Windows editions that support camera barcode scanner, including the camera requirements to successfully read barcodes.
Get started	Learn the basics for using camera barcode scanner.
Host a camera barcode scanner preview	Learn how to host a camera barcode scanner preview in your application.
Enable or disable the software decoder	Learn how to enable or disable the default software decoder.
Camera barcode scanner symbologies	Sample barcodes for each of the symbologies supported by the software barcode decoder that ships with Windows 10 Version 1803 or later (such as UPC/EAN, Code 39, Code 128, Interleaved 2 of 5, Databar Omnidirectional, Databar Stacked, QR Code, and GS1DWCCode).
Advanced configuration	Learn about advanced configuration options for camera barcode decoder.

See also

- [JustScanIt - Windows Store app](#) ↗
- [BarcodeScanner sample](#) ↗

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) ↗ | [Get help at Microsoft Q&A](#)

Camera barcode scanner system requirements

Article • 05/04/2023

[Camera barcode scanner](#) consists of a standard lens camera with auto-focus capabilities (attached to a computer) combined with a software decoder and is supported by Universal Windows Platform (UWP) apps on the following versions of Windows.

 Expand table

OS	Version
Windows 11	All
Windows 10	1803 (build 17134) or later

ⓘ Note

The software decoder built into Windows 10/11 is provided by [Digimarc Corporation](#) .

Webcam Requirements

 Expand table

Category	Recommendation	Comments
Focus	Auto Focus	Fixed focus is not recommended as it will have difficulty getting a clear view of the barcode due to dithering between the black and white elements of the barcode.
Resolution	1920 x 1440 or higher	We recommend cameras that are capable of 1920 x 1440 resolution or higher. Some lower resolution cameras can read standard barcodes if the barcode is printed large enough. Barcodes with thinner elements may require higher resolution cameras.

See also

- [JustScanIt - Windows Store app](#) 

- [BarcodeScanner sample](#) 

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

| [Get help at Microsoft Q&A](#)

Get started with a camera barcode scanner

Article • 05/04/2023

This topic describes how to set up a basic camera barcode scanner in a UWP application.

ⓘ Note

The software decoder built into Windows 10/11 is provided by [*Digimarc Corporation*](#) [↗].

The following code snippets are for demonstration purposes only. For a complete working sample, see the [Barcode scanner sample](#) [↗].

Step 1: Add capability declarations to your app manifest

1. In Microsoft Visual Studio, in **Solution Explorer**, open the designer for the application manifest by double-clicking the `package.appxmanifest` item.
2. Select the **Capabilities** tab.
3. Check the boxes for **Webcam** and **PointOfService**.

ⓘ Note

The **Webcam** capability is required for the software decoder to receive frames from the camera to both decode the barcode and provide a preview in your application.

Step 2: Add `using` directives

Csharp

```
using Windows.Devices.Enumeration;  
using Windows.Devices.PointOfService;
```

Step 3: Define your device selector

Use one of the [BarcodeScanner.GetDeviceSelector](#) methods to obtain a [BarcodeScanner](#) object for each connected barcode scanner.

Option A: Find all barcode scanners

Csharp

```
string selector = BarcodeScanner.GetDeviceSelector();
```

Option B: Find all barcode scanners based on scope (for this example, we filter on [Local](#) connection type)

Csharp

```
string selector =  
BarcodeScanner.GetDeviceSelector(PosConnectionTypes.Local);  
DeviceInformationCollection deviceCollection = await  
DeviceInformation.FindAllAsync(selector);
```

Step 4: Enumerate barcode scanners

If you do not expect the list of devices to change during the lifespan of your application, use [DeviceInformation.FindAllAsync](#) to get a one-time snapshot. However, if the list of barcode scanners could change during the lifespan of your application, use a [DeviceWatcher](#) instead.

Important

Using [GetDefaultAsync](#) to enumerate [PointOfService](#) devices can result in inconsistent behavior as it only returns the first device found in the class (which can change from session to session).

Option A: Enumerate a snapshot of all connected barcode scanners based on the *selector* created in Step 3

In this snippet, we create a [DeviceInformationCollection](#) object and use ********[DeviceInformation.FindAllAsync](#) to populate it.

Csharp

```
DeviceInformationCollection deviceCollection = await  
DeviceInformation.FindAllAsync(selector);
```

💡 Tip

See [Enumerate a snapshot of devices](#) for more information on using [DeviceInformation.FindAllAsync](#).

Option B: Enumerate available barcode scanners based on the *selector* created in Step 3 and watch for changes to that collection

In this snippet, we create a [DeviceWatcher](#) using [DeviceInformation.CreateWatcher](#).

Csharp

```
DeviceWatcher deviceWatcher = DeviceInformation.CreateWatcher(selector);  
watcher.Added += Watcher_Added;  
watcher.Removed += Watcher_Removed;  
watcher.Updated += Watcher_Updated;  
watcher.Start();
```

💡 Tip

For more information, see [Enumerate and watch devices](#) and [DeviceWatcher](#).

Step 5: Identify camera barcode scanners

A camera barcode scanner consists of a camera (attached to a computer) combined with a software decoder, which Windows dynamically pairs to create a fully functional [barcode scanner](#) for Universal Windows Platform (UWP) apps.

[BarcodeScanner.VideoDeviceId](#) can be used to differentiate between camera barcode scanners and physical barcode scanners. A non-NULL VideoDeviceId indicates that the barcode scanner object from your device collection is a camera barcode scanner. If you have more than one camera barcode scanner you might want to build a separate collection that excludes physical barcode scanners.

Camera barcode scanners using the decoder that ships with Windows are identified as:

Microsoft BarcodeScanner (*name of your camera here*)

If there is more than one camera, and they are built into the chassis of the computer, the name might differentiate between *front* and *rear* cameras.

When the *DeviceWatcher* starts (see [Step 4: Enumerate barcode scanners](#)), it enumerates through each connected device. In the following snippet, we add each available scanner to a *BarcodeScanner* collection and bind the collection to a *ListBox*.

C#

```
ObservableCollection<BarcodeScanner> barcodeScanners =  
    new ObservableCollection<BarcodeScanner>();  
  
private async void Watcher_Added(DeviceWatcher sender, DeviceInformation  
args)  
{  
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>  
    {  
        barcodeScanners.Add(new BarcodeScanner(args.Name, args.Id));  
  
        // Select the first scanner by default.  
        if (barcodeScanners.Count == 1)  
        {  
            ScannerListBox.SelectedIndex = 0;  
        }  
    });  
}
```

When the *SelectedIndex* of the *ListBox* changes (the first item is selected by default in the previous snippet), we query the device info (the *SelectScannerAsync* task is implemented in [Step 6: Claim the camera barcode scanner](#)).

C#

```
private async void ScannerSelection_Changed(object sender,  
SelectionChangedEventArgs args)  
{  
    var selectedScannerInfo = (BarcodeScanner)args.AddedItems[0];  
    var deviceId = selectedScannerInfo.DeviceId;  
  
    await SelectScannerAsync(deviceId);  
}
```

Step 6: Claim the camera barcode scanner

Call [BarcodeScanner.ClaimScannerAsync](#) to obtain exclusive use of the camera barcode scanner.

C#

```
private async Task SelectScannerAsync(string scannerDeviceId)
{
    selectedScanner = await BarcodeScanner.FromIdAsync(scannerDeviceId);

    if (selectedScanner != null)
    {
        claimedScanner = await selectedScanner.ClaimScannerAsync();
        if (claimedScanner != null)
        {
            await claimedScanner.EnableAsync();
        }
        else
        {
            rootPage.NotifyUser("Failed to claim the selected barcode scanner", NotifyType.ErrorMessage);
        }
    }
    else
    {
        rootPage.NotifyUser("Failed to create a barcode scanner object", NotifyType.ErrorMessage);
    }
}
```

Step 7: System provided preview

A camera preview is required to help the user aim the camera at a barcode. Windows provides a basic camera preview that launches a dialog to control of the camera barcode scanner.

Call [ClaimedBarcodeScanner.ShowVideoPreview](#) to open the dialog and [ClaimedBarcodeScanner.HideVideoPreview](#) to close it.

Tip

See [Hosting Preview](#) to host the preview for camera barcode scanner in your application.

Step 8: Initiate scan